



Get started Prepare ▾

Get full access



Blog Menu

Series Contents

50 React.js Interview Questions for Experienced Developers

50 React JS interview questions for experienced developers. Explore detailed solutions to enhance your preparation for 2025 job interviews

Advanced [#react](#) [#interviews](#)

**GreatFrontEnd Team**

21 min read Jun 22, 2025

Copy link

Share

50 React.js Interview Questions for Experienced Developers

For experienced frontend developers, React interviews often focus on advanced concepts that evaluate both problem-solving abilities and architectural knowledge. With over a decade of React experience, you're expected to have a strong grasp of the framework's core principles, as well as its more sophisticated patterns and performance optimization strategies.

To help you shine in these interviews, we've compiled a list of 50 React JS interview questions. These questions cover everything from React's lifecycle and hooks to context management, error boundaries, and performance bottlenecks, ensuring you're well-prepared to demonstrate your expertise in handling complex challenges and building scalable applications.

If you're looking for more in-depth React interview preparation materials, also check out these resources:

- [React Interview Playbook](#)
- [Practice React coding interview questions](#)
- [Practice React quiz interview questions](#)

1. Can you explain how React's Virtual DOM works and its benefits?

The Virtual DOM (VDOM) is a lightweight, in-memory representation of the real DOM. When a React component's state or props change, React first updates the VDOM instead of the actual DOM. Then, React performs a "diffing" algorithm to calculate the minimum number of changes required to update the real DOM, ensuring efficient UI rendering.

Benefits:

- Performance: Minimizes direct DOM manipulations, which are expensive.
- Declarative UI: Developers describe "what" the UI should look like, and React manages "how" to update it.
- Cross-browser Compatibility: React abstracts browser quirks when manipulating the DOM.

```
1  function Counter() {  
2    const [count, setCount] = React.useState(0);  
3  
4    return (  
5      <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>  
6    );  
7  }
```

[Read more about it here](#)

2. How does React's reconciliation algorithm work to update the UI efficiently?

React's reconciliation algorithm identifies changes in the VDOM tree and updates only the affected parts of the real DOM. It uses the following steps:

1. Diffing: Compares the previous and current VDOM tree to find changes.
2. Key Comparison: Uses `key` props to identify stable elements across renders.

3. Batch Updates: Minimizes updates by batching multiple state changes into a single DOM operation.

3. What is the difference between React's class components and functional components?

Class Components

- Use ES6 classes
- Manage state and lifecycle using class methods like `componentDidMount`
- Verbose and harder to test

Functional Components

- Simpler functions.
- Use hooks (`useState` , `useEffect` , etc.) for state and lifecycle
- Encouraged in modern React due to their simplicity and performance benefits

4. Explain the Fiber architecture and how it improves React's rendering process.

The Fiber architecture is a re-implementation of React's reconciliation algorithm designed to improve rendering. It breaks rendering into units of work that can be paused and resumed, allowing React to prioritize high-priority tasks (e.g., user input).

Benefits:

- Non-blocking rendering
- Improved responsiveness for animations and transitions

5. What are React fragments, and when would you use them instead of a wrapper element?

React Fragments allow grouping child elements without adding an extra DOM node.

When to use:

- Avoiding unnecessary `<div>` elements in the DOM, which can cause layout or CSS issues

```
1  function List() {  
2    return (  
3      <>  
4        <li>Item 1</li>  
5        <li>Item 2</li>  
6      </>  
7    );  
8  }
```

[Read more about it here](#)

6. What is the difference between props and state in React?

- **Props** are immutable and passed from a parent component to a child component. They allow components to be dynamic by providing external values.
- **State** is mutable and managed within the component itself. It can change over time, usually in response to user actions or network responses. Example:

```
1  // Parent component passes props  
2  const Parent = () => {  
3    return <Child name="John" />;  
4  };  
5  
6  // Child component receives props
```

```
7   const Child = ({ name }) => {  
8     return <h1>Hello, {name}</h1>;  
9   };
```

[Read more about it here](#)

7. How would you lift state up in a React application, and why is it necessary?

Lifting state up means moving the state to the nearest common ancestor of the components that need to access it. This is necessary for sharing state between sibling components.

Example:

```
1  // Lifting state up  
2  const Parent = () => {  
3    const [counter, setCounter] = useState(0);  
4  
5    return (  
6      <div>  
7        <Child1 counter={counter} />  
8        <Child2 setCounter={setCounter} />  
9      </div>  
10   );  
11 };  
12  
13 const Child1 = ({ counter }) => <h1>{counter}</h1>;  
14 const Child2 = ({ setCounter }) => (  
15   <button onClick={() => setCounter((prev) => prev + 1)}>Increment</button>  
16 );
```

In this example, the state is managed in the `Parent` component, and both child components access it via props.

8. Can you explain how you would manage deeply nested state in React?

To manage deeply nested state, you can use one of the following techniques:

- `useState` : For simpler state updates in nested structures.
- `useReducer` : More useful for complex or deeply nested state, where actions and reducers help keep state changes predictable.
- Context API: For shared deep state across multiple components.
- Immutable updates: Use libraries like Immer for easier state updates when working with nested data. Example with `useReducer`:

```
1  const initialState = { user: { name: 'John', age: 30 } };
2
3  function reducer(state, action) {
4    switch (action.type) {
5      case 'UPDATE_NAME':
6        return { ...state, user: { ...state.user, name: action.payload } };
7      default:
8        return state;
9    }
10 }
11
12 const Component = () => {
13   const [state, dispatch] = useReducer(reducer, initialState);
14
15   return (
16     <button onClick={() => dispatch({ type: 'UPDATE_NAME', payload: 'Jane' })}>
17       Update Name
18     </button>
19   );
20 };
```

9. How do controlled and uncontrolled components differ in React?

- **Controlled Components:** React is responsible for managing the form element's value via state. Any change to the value is handled by React through the onChange handler.
- **Uncontrolled Components:** React does not manage the form value directly. Instead, the DOM itself handles the state, and React uses refs to access the current value. Example:

```
1  // Controlled component
2  const ControlledInput = () => {
3    const [value, setValue] = useState('');
4    return <input value={value} onChange={(e) => setValue(e.target.value)} />;
5  };
6
7  // Uncontrolled component
8  const UncontrolledInput = () => {
9    const inputRef = useRef();
10   return <input ref={inputRef} />;
11  };
```

[Read more about it here](#)

10. What are some common methods to prevent unnecessary re-renders of a component in React?

Here are some ways to prevent unnecessary re-renders:

- **React.memo:** Memoizes a component, preventing re-renders when props haven't changed.
- **useMemo:** Memoizes values, useful for expensive calculations.
- **useCallback:** Memoizes functions, ensuring that they don't get recreated on every render.
- **shouldComponentUpdate:** A lifecycle method that determines if a component should re-render (for class components).
- **PureComponent:** Extends React.Component and only re-renders when props or state change.

- **Key Prop in Lists:** Using a proper key for list items can help React efficiently update only the necessary DOM elements. Example:

```
1  // Using React.memo
2  const MyComponent = React.memo(({ name }) => {
3    return <h1>{name}</h1>;
4  });
5
6  // Using useMemo
7  const computedValue = useMemo(() => expensiveComputation(a, b), [a, b]);
8
9  // Using useCallback
10 const memoizedCallback = useCallback(() => {
11   console.log('This function is memoized');
12 }, []);
```

11. How do hooks improve React components?

Hooks allow using state and lifecycle features in functional components, making them concise, reusable, and easier to test.

12. What is the purpose of the `useEffect` hook, and how do you manage dependencies in it?

The `useEffect` hook handles side effects like fetching data, updating the DOM, or setting up subscriptions in functional components. Dependencies, specified as the second argument (`[]`), control when the effect runs:

- Empty array (`[]`): Runs only after the initial render
- No array provided: Runs after every render
- Specific dependencies (`[dep1, dep2]`): Runs when any dependency changes

[Read more about it here](#)

13. What is the difference between `useMemo` and `useCallback` ?

The key difference between `useMemo` and `useCallback` lies in what they cache:

- `useMemo` : Caches the result of a computation to avoid re-computing it unless its dependencies change. Useful for optimizing expensive calculations.
- `useCallback` : Caches a function instance to avoid unnecessary re-creation unless its dependencies change. Useful for preventing child components from re-rendering when passed as props.

```
const memoizedValue = useMemo(() => computeValue(a, b), [a, b]);  
const memoizedCallback = useCallback(() => callback(a, b), [a, b]);
```

[Read more about it here](#)

14. What is the `useReducer` hook in React and when should it be used?

The `useReducer` hook manages complex state logic in functional components, serving as an alternative to `useState`. It's ideal when state has multiple sub-values or when the next state relies on the previous one. It accepts a reducer function and an initial state.

```
const [state, dispatch] = useReducer(reducer, initialState);
```

[Read more about it here](#)

15. What is the difference between `useEffect` and `useLayoutEffect` in React?

`useEffect` and `useLayoutEffect` are both used for handling side effects in React functional components but differ in timing:

- `useEffect` runs asynchronously after the DOM has painted, ideal for tasks like data fetching or subscriptions.
- `useLayoutEffect` runs synchronously after DOM mutations but before the browser paints, useful for tasks like measuring DOM elements or synchronizing the UI with the DOM.

Example:

```
1  import React, { useEffect, useLayoutEffect, useRef } from 'react';
2
3  function Example() {
4    const ref = useRef();
5
6    useEffect(() => {
7      console.log('useEffect: Runs after DOM paint');
8    });
9
10   useLayoutEffect(() => {
11     console.log('useLayoutEffect: Runs before DOM paint');
12     console.log('Element width:', ref.current.offsetWidth);
13   });
14
15   return <div ref={ref}>Hello</div>;
16 }
```

[Read more about it here](#)

16. Explain the React component lifecycle methods in class components.

React class components have lifecycle methods for different phases:

Mounting:

- `constructor` : Initializes state or binds methods
- `componentDidMount` : Runs after the component mounts, useful for API calls or subscriptions

```
componentDidMount() {  
  console.log('Component mounted');  
}
```

Updating:

- `shouldComponentUpdate` : Determines if the component should re-render
- `componentDidUpdate` : Runs after updates, useful for side effects

Unmounting:

- `componentWillUnmount` : Cleans up (e.g., removing event listeners).

```
componentWillUnmount() {  
  console.log('Component will unmount');  
}
```

These methods allow you to manage component behavior throughout its lifecycle.

17. How do `shouldComponentUpdate` and `React.memo` optimize re-renders?

- `shouldComponentUpdate` : A lifecycle method in class components. Return false to prevent unnecessary renders.
- `React.memo` : A higher-order component for functional components to prevent re-renders unless props change.

```
const MemoizedComponent = React.memo(({ prop }) => <div>{prop}</div>);
```

18. What is the purpose of the `key` prop in lists, and how does React use it to optimize rendering?

The `key` prop uniquely identifies elements in a list, enabling React to efficiently update the DOM by matching keys during the reconciliation process. Without unique keys, React may unnecessarily re-render elements, leading to performance issues and bugs.

```
{
  items.map((item) => <ListItem key={item.id} value={item.value} />);
}
```

[Read more about it here](#)

19. Can you explain how to use `React.PureComponent` and when it should be used?

`React.PureComponent` is a class component that performs a shallow comparison of props and state to determine if the component should re-render. Use it when the component's render output depends solely on its props and state.

20. How do you avoid or handle performance bottlenecks in a large-scale React application?

- Code Splitting: Use `React.lazy` and `Suspense` to load components only when needed, improving initial load time
- Memoization: Use `useMemo` and `React.memo` to memoize expensive calculations and components, preventing unnecessary re-renders

- Debouncing/Throttling: Limit frequent updates (like search inputs) with debouncing or throttling to optimize performance
- Pagination and Infinite Scroll: Load data in chunks with pagination or infinite scroll to reduce rendering large datasets at once

21. What are higher-order components in React?

Higher-order components (HOCs) are functions that take a component and return a new one with added props or behavior, facilitating logic reuse across components.

```
1  const withExtraProps = (WrappedComponent) => {  
2    return (props) => <WrappedComponent {...props} extraProp="value" />;  
3  };  
4  
5  const EnhancedComponent = withExtraProps(MyComponent);
```

[Read more about it here](#)

22. Explain how React's Context API works and when to use it

The Context API allows sharing state across the component tree without prop drilling. You create a context with `React.createContext()`, wrap your app with a `Provider` to supply the state, and access it with `useContext()` in any component.

When to use:

Use the Context API for global state (like themes, user data) that needs to be accessed by multiple components, avoiding prop drilling. Example:

```
1  const ThemeContext = React.createContext();
```

```
2  const App = () => {  
3    const [theme, setTheme] = useState('light');  
4    return (  
5      <ThemeContext.Provider value={{ theme, setTheme }}>  
6        <SomeComponent />  
7      </ThemeContext.Provider>  
8    );  
9  };
```

23. What are the benefits and limitations of using React's Context API for state management?

Benefits:

- Simplifies State Sharing: Allows easy sharing of state across components without prop drilling.
- Eliminates Prop Drilling: Avoids passing props through multiple layers, making the code more maintainable.

Limitations:

- Inefficient for Frequent Updates: Re-renders all components that consume the context on state changes, which can be inefficient for frequent updates.
- Not Suitable for Large-Scale State Management: For complex or large applications with deep state needs, consider using state management libraries like Redux or Zustand for better performance and scalability.

[Read more about it here](#)

24. How do you implement lazy loading in React, and what is the role of `React.lazy` and `Suspense` ?

`React.lazy` dynamically loads a component only when needed. `Suspense` displays a fallback UI while loading.

```
1  const LazyComponent = React.lazy(() => import('./LazyComponent'));
2
3  function App() {
4    return (
5      <Suspense fallback={<div>Loading...</div>}>
6        <LazyComponent />
7      </Suspense>
8    );
9  }
```

[Read more about it here](#)

25. Can you explain how to create and use custom hooks in React?

Custom hooks are functions that encapsulate reusable logic.

```
1  function useCounter(initialValue = 0) {
2    const [count, setCount] = React.useState(initialValue);
3    const increment = () => setCount((prev) => prev + 1);
4    return { count, increment };
5  }
```

26. How do you manage forms in React?

Controlled Components:

Use `useState` to manage form data, where the form elements' values are controlled by React state. This allows for easy validation and manipulation of form inputs.

```
const [value, setValue] = useState('');
const handleChange = (e) => setValue(e.target.value);
return <input value={value} onChange={handleChange} />;
```

Uncontrolled Components:

Use `useRef` to directly access DOM elements and their values, which can be useful for simple forms or when you want to avoid managing state.

```
const inputRef = useRef();
const handleSubmit = () => console.log(inputRef.current.value);
return <input ref={inputRef} />;
```

Controlled components offer more flexibility and control, while uncontrolled components can be simpler and more efficient for certain use cases.

27. What is the role of `useRef` in form validation, and how would you implement it?

`useRef` allows direct access to DOM elements, which is useful for form validation in uncontrolled components or when you need to interact with form fields without managing their state. It can also be used to store values or functions without causing re-renders.

Implementation: You can use `useRef` to reference input elements and then access their values for validation when the form is submitted.

```
1  const inputRef = useRef();
2
```



```
3  function handleSubmit() {  
4    const value = inputRef.current.value;  
5    if (value.trim() === '') {  
6      console.log('Input is required!');  
7    } else {  
8      console.log('Input value:', value);  
9    }  
10 }  
11  
12 return <input ref={inputRef} />;
```

28. How would you handle form validation in React?

For robust form validation, use libraries like Formik or React Hook Form, which handle validation and error management efficiently.

For simple forms, use `useState` to track input values and validation errors:

```
1  const [value, setValue] = useState('');  
2  const [error, setError] = useState('');  
3  
4  const handleSubmit = () => {  
5    if (!value) setError('Field is required');  
6    else console.log('Form submitted:', value);  
7  };
```

Use libraries for complex forms, and `useState` for basic validation.

29. How do you handle dynamic form fields in React?

To handle dynamic form fields, use state arrays to manage and update the fields as they are added or removed.

Example:

```
1  const [fields, setFields] = useState(['']);
2
3  const addField = () => setFields([...fields, '']);
4  const removeField = (index) => setFields(fields.filter((_, i) => i !== index));
5
6  return (
7    <>
8      {fields.map((_, index) => (
9        <input key={index} />
10      ))}
11    <button onClick={addField}>Add Field</button>
12  </>
13  );
```

This approach allows you to dynamically add or remove form fields while keeping the state updated.

30. Can you explain the debouncing technique for form inputs in React?

Debouncing delays execution of an action (e.g., API call) until a specified time has elapsed since the last input.

```
const handleInput = debounce((value) => console.log(value), 300);
```

31. How would you test a React component using Jest and React Testing Library?

- Write tests to ensure proper rendering, interaction, and output.
- Use `render`, `fireEvent`, and `screen` for assertions.

```
test('renders button', () => {  
  render(<button>Click Me</button>);  
  expect(screen.getByText('Click Me')).toBeInTheDocument();  
});
```

32. What is the purpose of snapshot testing in React?

Snapshot testing captures a component's rendered output (DOM structure) at a specific point in time and compares it to a saved baseline to detect unintended changes. It helps ensure UI consistency by alerting you to unexpected modifications.

33. How do you handle asynchronous testing in React components?

Use `async/await` and `waitFor` from React Testing Library to wait for updates after async operations.

Example:

```
1  import { render, screen, waitFor } from '@testing-library/react';  
2  import MyComponent from './MyComponent';  
3  
4  test('displays data after fetching', async () => {  
5    render(<MyComponent />);  
6    await waitFor(() =>  
7      expect(screen.getByText(/fetched data/i)).toBeInTheDocument(),  
8    );  
9  });
```

This ensures the test waits for async updates before making assertions.

34. What are the key differences between shallow rendering and full DOM rendering in React tests?

- **Shallow Rendering:** Renders only the component being tested, without rendering its child components. Useful for isolated unit testing.
- **Full DOM Rendering:** Mounts the entire component tree, including children, providing a complete DOM structure. Ideal for integration tests.

35. How would you mock API calls in a React test?

Mock API calls using Jest's `jest.mock` or libraries like Axios Mock Adapter to simulate responses and test components in isolation.

Example:

```
1  import axios from 'axios';
2  import { render, screen, waitFor } from '@testing-library/react';
3  import MyComponent from './MyComponent';
4
5  jest.mock('axios');
6
7  test('displays fetched data', async () => {
8    axios.get.mockResolvedValue({ data: { message: 'Hello, World!' } });
9
10   render(<MyComponent />);
11   await waitFor(() =>
12     expect(screen.getByText('Hello, World!')).toBeInTheDocument(),
13   );
14 });
```

Key Points: - `jest.mock` : Replaces the actual module with a mock. 0 Mock Responses: Use `mockResolvedValue` or `mockRejectedValue` to simulate API behavior.

36. How does React Router work, and how do you implement dynamic routing?

React Router maps URL paths to components, enabling navigation in single-page apps. Dynamic routing allows you to use URL parameters to render components based on dynamic values.

Example:

```
1  import { BrowserRouter, Routes, Route, useParams } from 'react-router-dom';
2
3  function UserPage() {
4    const { id } = useParams(); // Access dynamic parameter
5    return <h1>User ID: {id}</h1>;
6  }
7
8  export default function App() {
9    return (
10     <BrowserRouter>
11       <Routes>
12         <Route path="/user/:id" element={<UserPage />} /> {/* Dynamic path */}
13       </Routes>
14     </BrowserRouter>
15   );
16 }
```

Key Features:

- Dynamic Segments: `:id` captures dynamic data from the URL.
- `useParams` Hook: Accesses these dynamic values for rendering.

37. How do you handle nested routes and route parameters in React Router?

Nested routes allow you to create hierarchies of components, and `useParams` helps access dynamic route parameters.

Key Techniques:

- `<Outlet>` : Renders child routes within a parent layout
- `useParams` : Retrieves route parameters for dynamic routing

```
1  import {
2    BrowserRouter,
3    Routes,
4    Route,
5    Outlet,
6    useParams,
7  } from 'react-router-dom';
8
9  function UserProfile() {
10    const { userId } = useParams();
11    return <h2>User ID: {userId}</h2>;
12  }
13
14  function App() {
15    return (
16      <BrowserRouter>
17        <Routes>
18          <Route path="user/:userId" element={<Outlet />}>
19            <Route path="profile" element={<UserProfile />} />
20          </Route>
21        </Routes>
22      </BrowserRouter>
23    );
24  }
```

38. What is the difference between BrowserRouter and HashRouter?

- **BrowserRouter:** Uses the HTML5 History API to manage navigation, enabling clean URLs without the hash (#). It requires server-side configuration to handle routes correctly, especially for deep linking.

- **HashRouter:** Uses the hash (#) portion of the URL to simulate navigation. It doesn't require server-side configuration, as the hash is never sent to the server. This makes it suitable for environments where server-side routing isn't possible (e.g., static hosting).

39. How would you implement route guards or private routes in React?

To implement private routes, create a component that checks if the user is authenticated before rendering the desired route.

Example:

```
1  import { Navigate } from 'react-router-dom';
2
3  function PrivateRoute({ children }) {
4    return isAuthenticated ? children : <Navigate to="/login" />;
5  }
```

- **PrivateRoute** : Checks authentication and either renders the children (protected routes) or redirects to the login page.
- **<Navigate>** : Replaces the deprecated **<Redirect>** for redirecting in React Router v6+.

40. How do you manage the active route state in a multi-page React application?

Use the `useLocation` hook to get the current route, and conditionally apply styles for the active state.

Example:

```
1  import { useLocation } from 'react-router-dom';
```

```
2
3  function NavBar() {
4    const location = useLocation();
5    return (
6      <nav>
7        <ul>
8          <li className={location.pathname === '/home' ? 'active' : ''}>Home</li>
9          <li className={location.pathname === '/about' ? 'active' : ''}>
10            About
11          </li>
12        </ul>
13      </nav>
14    );
15  }
```

41. What are error boundaries in React, and how do they help handle errors in the component tree?

Error boundaries are React components that catch JavaScript errors anywhere in the component tree and log those errors, preventing the entire app from crashing. They display a fallback UI instead of the component tree that crashed.

```
1  class ErrorBoundary extends React.Component {
2    state = { hasError: false };
3
4    static getDerivedStateFromError() {
5      return { hasError: true };
6    }
7
8    componentDidCatch(error, info) {
9      console.log(error, info);
10   }
11
12   render() {
13     if (this.state.hasError) {
14       return <h1>Something went wrong!</h1>;
15     }
16     return this.props.children;
```



```
17   }  
18 }
```

[Read more about it here](#)

42. Can you explain how to implement global error handling in a React application?

To implement global error handling, you can use Error Boundaries at a top level in your application (such as wrapping your entire app or specific routes). This ensures that even if a component crashes, the rest of the app continues to function.

43. How do you handle asynchronous errors in React?

For asynchronous errors (like API calls), use `try-catch` within `async` functions, and handle them using state to show error messages. You can also integrate error boundaries to catch these errors in components.

```
1  const fetchData = async () => {  
2    try {  
3      const data = await fetch('/api/data');  
4      const result = await data.json();  
5    } catch (error) {  
6      setError(error.message);  
7    }  
8  };
```

44. How would you show a fallback UI in case of errors or slow network requests?

Use React Suspense for async components, and display a fallback UI such as a loading spinner or error message when the component or data is still loading.

```
<Suspense fallback={<Loading />}>  
  <MyComponent />  
</Suspense>
```

You can also combine error boundaries with loading states to handle network failures.

45. Can you describe a strategy for handling large-scale error logging and monitoring in React?

For large-scale applications, use external error logging services such as **Sentry** or **LogRocket**. Integrate these tools into your app to automatically capture and monitor errors in production. Additionally, maintain custom error boundaries and provide meaningful error messages for debugging.

46. What are React portals, and in what scenarios would you use them?

React portals provide a way to render children outside the parent component's DOM hierarchy. They are useful for modals, tooltips, or any UI element that needs to break out of the regular DOM flow but remain part of the React tree.

```
ReactDOM.createPortal(<Modal />, document.getElementById('modal-root'));
```

[Read more about it](#)

47. Can you explain the concept of React Suspense, and how does it help in data fetching?

React Suspense is a feature that allows components to "wait" for something (like data or a code-split chunk) before rendering. It improves user experience by showing a fallback UI until the data is ready.

```
<Suspense fallback={<Loading />}>  
  <DataFetchingComponent />  
</Suspense>
```

48. What is Concurrent Mode in React, and how does it improve rendering performance?

Concurrent Mode allows React to work on multiple tasks simultaneously without blocking the main UI thread. It enables React to prioritize updates and provide smoother rendering for complex applications.

49. How does React handle concurrent rendering with multiple updates and prioritize them?

React uses the priority system in Concurrent Mode to schedule updates. It can break up large updates into smaller chunks and give priority to user interactions (like clicks or input) to ensure the app remains responsive.

50. How would you handle long-running tasks or expensive computations in React applications without blocking the UI?

To avoid blocking the UI, use **Web Workers**, `setTimeout`, or `requestIdleCallback` for offloading heavy computations. Alternatively, break tasks into smaller parts and use React's

Suspense or useMemo to only recompute when necessary.

Example using `setTimeout` for deferring computation:

```
1  const [data, setData] = useState(null);
2
3  useEffect(() => {
4    setTimeout(() => {
5      const result = computeExpensiveData();
6      setData(result);
7    }, 0);
8  }, []);
```

[Copy link](#)[Share](#)[← Previous](#)[Next →](#)

**Advanced JavaScript Interview
Questions for 10+ Years of Experience**

**Angular Interview Questions for
Experienced**

Related articles



30 Basic to Advanced React Interview Questions with Solutions

30 Basic to Advanced React Interview Questions with Solutions

Jul 1, 2025 [#react](#)

30 React interview questions and solutions, covering basic to advanced topics. Ideal for developers preparing for their next job interview in 2025



Subscribe to our newsletter

Enter your email

Notify me

Practice

Get started

JavaScript functions

User interface coding

System design

Quiz

Guides

Front End Interview Playbook

Front End System Design Playbook

React Interview Playbook

Behavioral Interview Playbook

Study Plans

1 Week Plan

1 Month Plan

3 Months Plan

Company

Pricing

Promotions

Roadmap

About

Team

Contact us

Advertise with us New

Become an affiliate

Careers

Blog

Medium

DEV Community



English (US) ▾

© 2026 Codeney Pte Ltd. All rights reserved.

[Privacy Policy](#)

[Terms of Service](#)